Introduction to Big Data
DS-02 Assignment 2
Permiakov Lev
1. Methodology

1.1 System Architecture

The system is implemented as a distributed search engine consisting of three main stages: data preparation using Apache Spark, index construction using Hadoop MapReduce and HDFS, and query processing using Spark and ScyllaDB. HDFS is used as the primary storage during the indexing phase, while ScyllaDB is used to store the index for efficient query-time access. This separation allows batch processing for indexing and low-latency access for search operations.

1.2 Data Preparation and Transfer to HDFS

The input dataset is provided in Parquet format (a.parquet). Data preprocessing is performed using Spark. The system extracts the fields id, title, and text, samples approximately 1000 documents, and converts each document into a separate .txt file. Each file is named using the format doc_id_title.txt, which allows direct extraction of metadata during indexing.

After preprocessing: the Parquet file is uploaded to HDFS, generated text documents are uploaded to /data in HDFS. This step is required because MapReduce operates only on data stored in HDFS.

1.3 Index Construction (MapReduce)

The inverted index is built using a two-stage MapReduce pipeline. During the first stage, each document is tokenized by converting text to lowercase and removing punctuation. Term frequency (TF) is computed for each term within a document, and the document length is calculated. The reducer aggregates these values for each (term, document) pair. During the second stage, the inverted index is constructed. For each term, a postings list is created containing all documents where the term appears, along with term frequency and document length. At the same time, the document frequency (df) is computed. Additionally, global statistics are calculated: total number of documents (N), average document length (avgdl). The final index is stored in HDFS under /index/final.

1.4 Data Transfer from HDFS to ScyllaDB
After the index is built, it is transferred from HDFS to ScyllaDB. The system reads index files from HDFS, parses them line by line, and inserts the data into the database using a Python script. This step converts the batch-processed index into a format optimized for fast query execution.

1.5 Database Structure (ScyllaDB)
ScyllaDB is used as the storage layer.The database consists of four tables.
The docs table stores document metadata:
doc_id (primary key)
length
title

The vocabulary table stores terms and their document frequency:
term (primary key)
df

The postings table represents the inverted index:
term (partition key)
doc_id (clustering key)
tf
dl

The stats table stores global statistics:
name (primary key)
value
This schema is chosen because it directly reflects the structure of an inverted index. Using the term as a partition key allows efficient retrieval of all documents containing a given term.

1.6 Query Processing
Query processing is implemented using Spark.
The workflow is as follows:
1. the query is tokenized
2. postings, vocabulary, documents, and statistics are loaded from ScyllaDB
3. relevance scores are computed
4. scores are aggregated per document

5. results are sorted and top 10 documents are returned

1.7 Ranking Algorithm (BM25)

The system uses the BM25 ranking function.BM25 is computed as:

$BM25 = \Sigma\, IDF(t) * (tf * (k1 + 1)) / (tf + k1 * (1 - b + b * dl / avgdl))$

IDF is defined as:

$IDF(t) = \log((N - df + 0.5) / (df + 0.5) + 1)$

Where:

tf — term frequency in the document

df — number of documents containing the term

dl — document length

avgdl — average document length

N — total number of documents

Parameters used:

k1 = 1

b = 0.75

1.8 Computation Algorithm

The ranking process follows these steps:

1)Retrieve postings for all query terms

2)Compute partial BM25 scores for each (term, document) pair

3)Aggregate scores per document using reduceByKey

4)Sort results in descending order

5)Return top 10 documents

2. Demonstration

2.1 System Execution

The system is executed using Docker:

`docker compose up`

This command starts: Hadoop (HDFS + YARN), ScyllaDB, the full processing pipeline

2.2 Execution Pipeline

The system performs the following steps:

1)Upload dataset to HDFS

2)Generate text documents using Spark

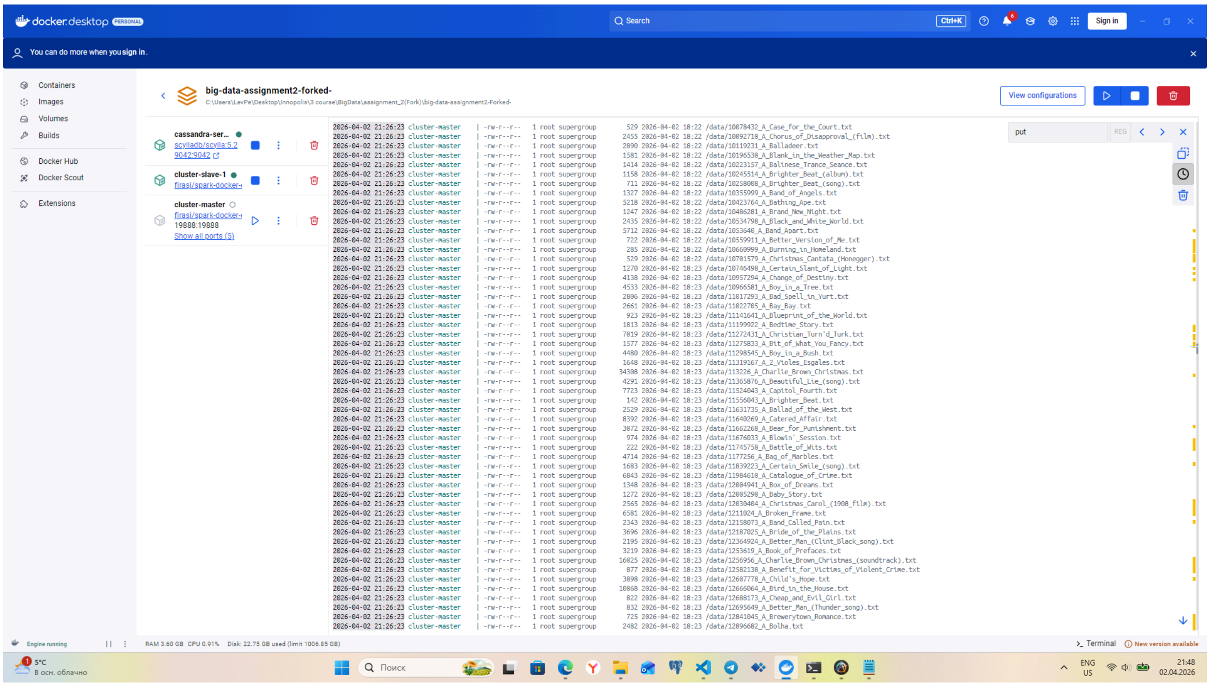
3)Upload documents to /data

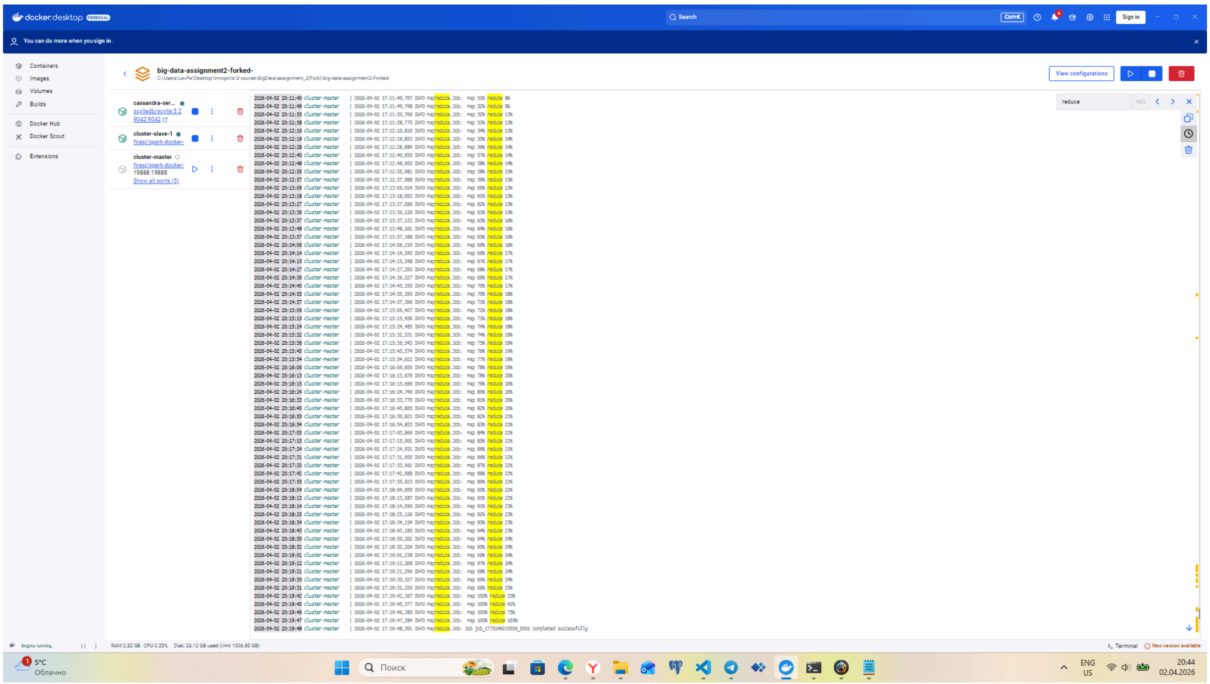
4)Build index using MapReduce
5)Store index in /index/final
6)Transfer index to ScyllaDB
7)Execute search query

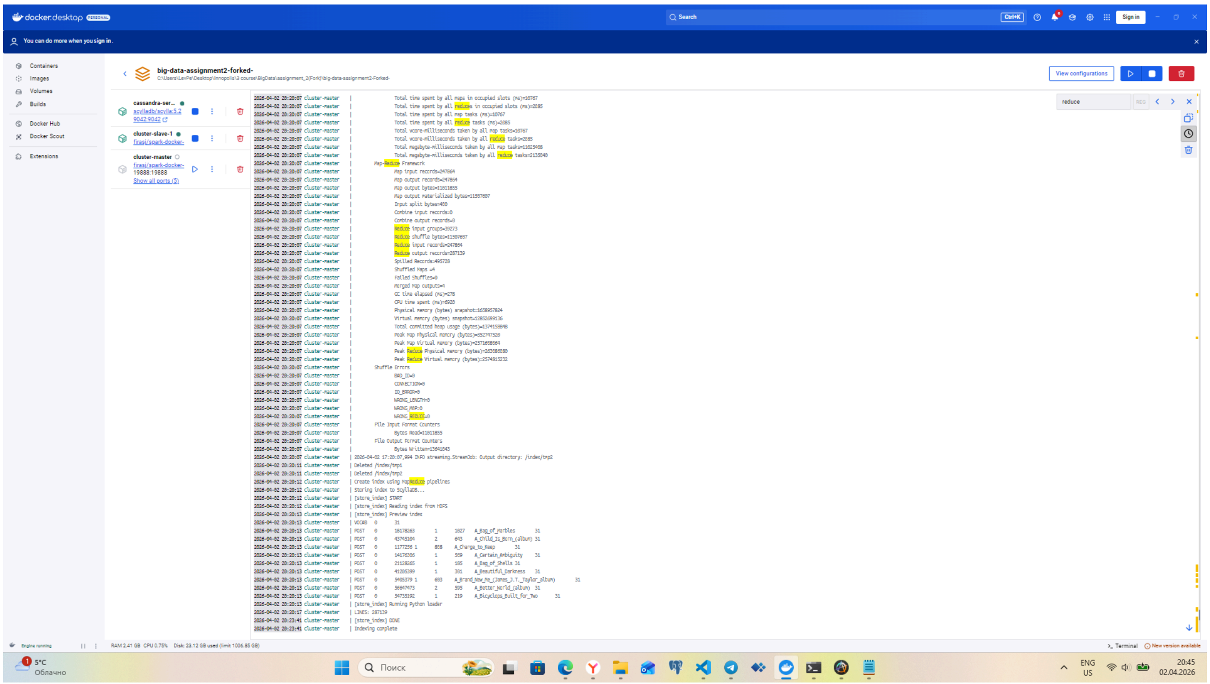
2.3 Indexing Results

After execution:

1. Documents are stored in /data
2. Index is stored in /index/final
3. Data is loaded into ScyllaDB







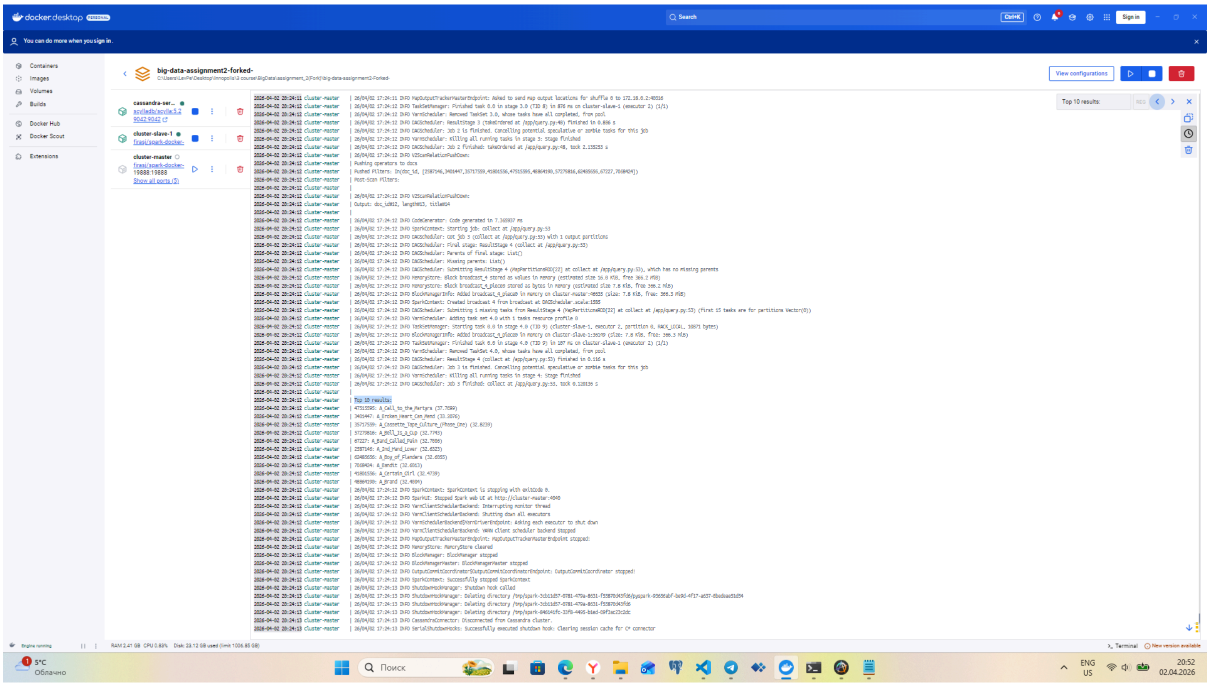
2.4 Query Execution

Output:

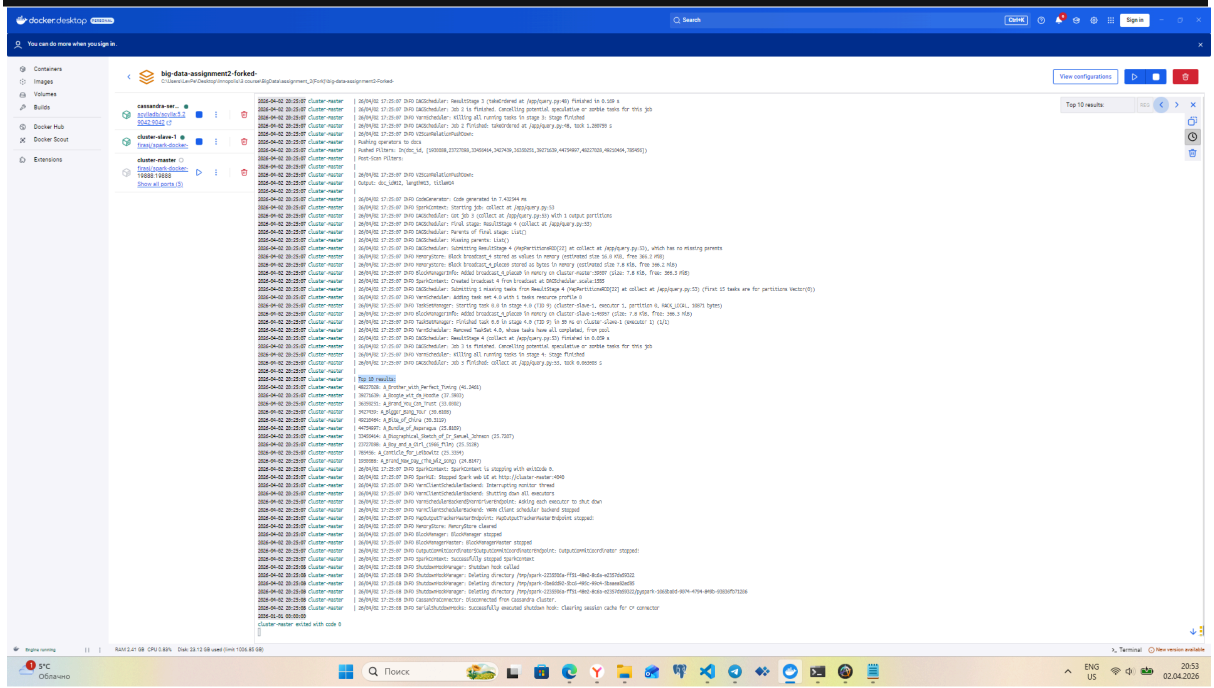
Top 10 documents

Each with BM25 score

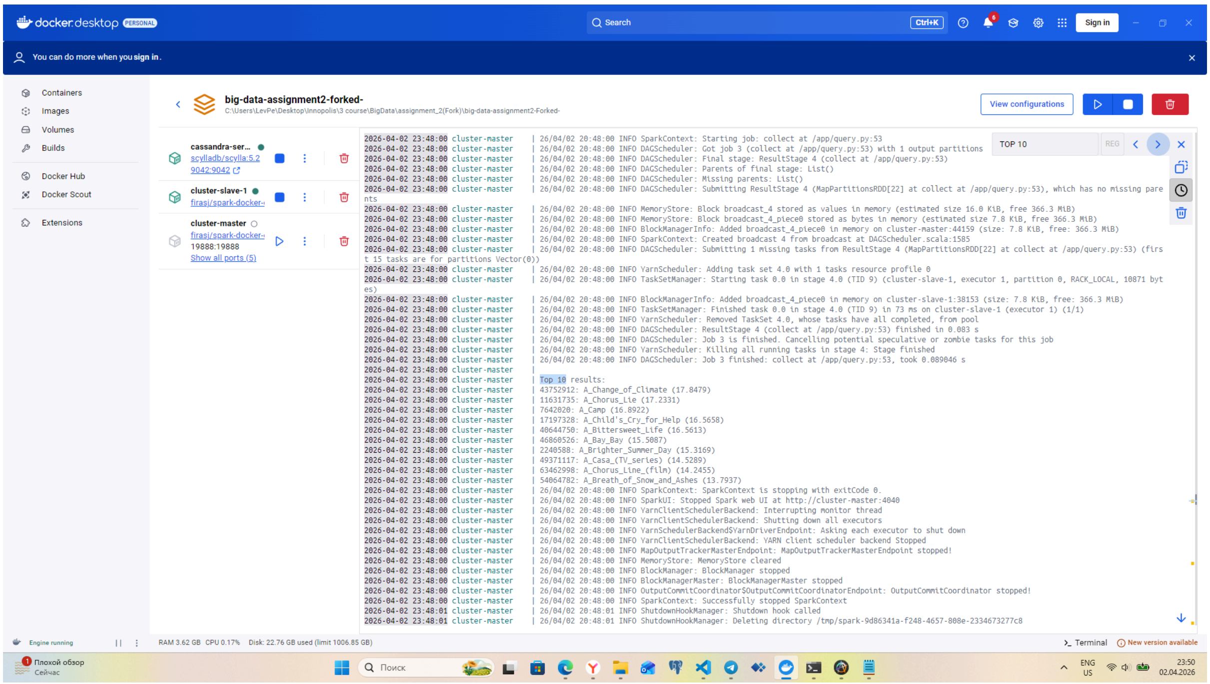
```
"dog play with cat"
```



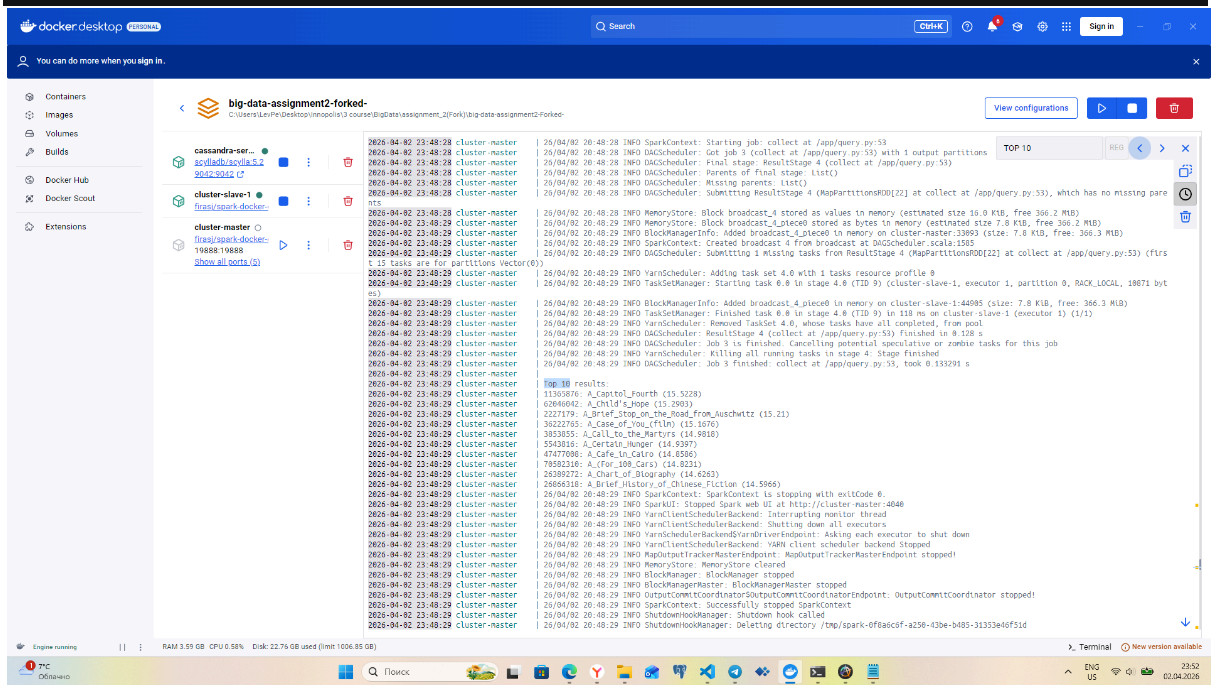
"big data and spark"



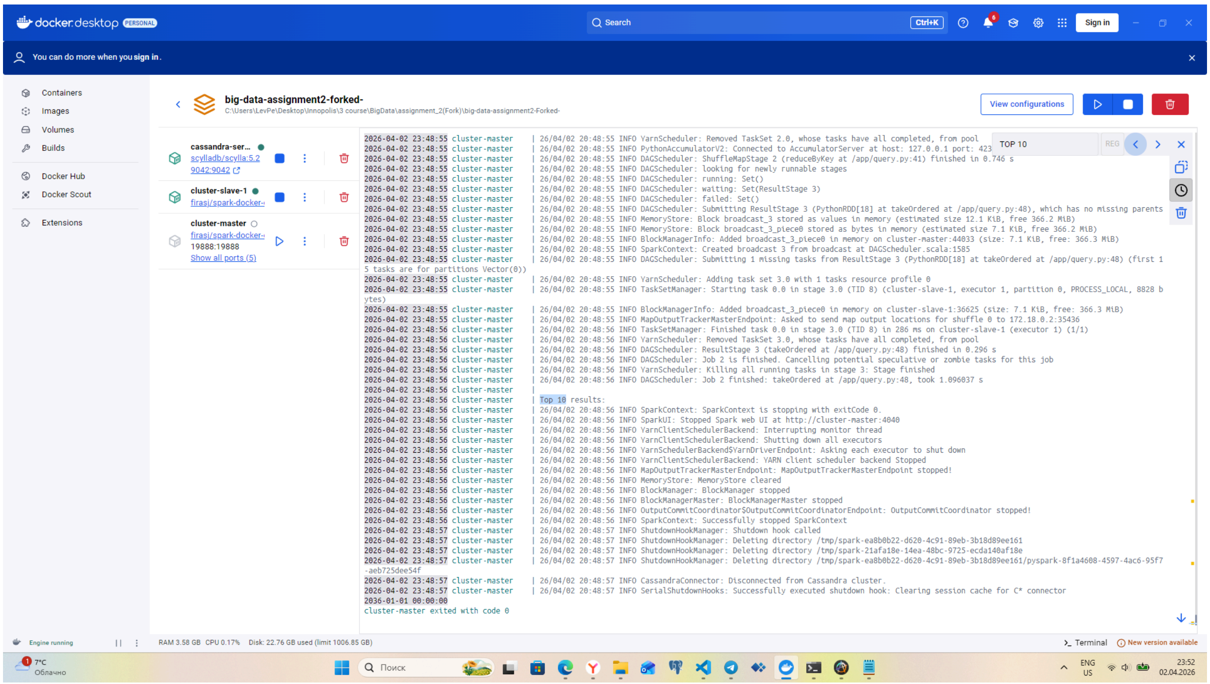
"ballad"



"beautiful"



"Hadoop"



2.5 Summary

The system implements a full search engine pipeline:

1. Inverted index construction using MapReduce
2. Storage in ScyllaDB
3. Query processing using Spark
4. Ranking using BM25

Explanation

When analyzing search queries, the following patterns are observed: for multi-word queries consisting of rare terms such as "dog play with cat" and "big data and spark", the total BM25 score ranges from 35 to 40, which is explained by the summation of high IDF values of individual terms. However, the actual search results do not match the topic of the queries due to the absence of such words in the document corpus. For single-word queries like "ballad" or "beautiful", the scores are predictably lower — in the range of 13 to 18 — since these words are present in the index, and the retrieved results are indeed relevant. In the case of the query "hadoop", there is no top result set at all, because this word does not appear in any of the 1000 indexed documents, which is the correct behavior of the search system.